



On the fast computation of the Dirichlet-multinomial log-likelihood function

Alessandro Languasco¹ · Mauro Migliardi²

Received: 21 January 2022 / Accepted: 4 December 2022

© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2022

Abstract

We introduce a new algorithm to compute the difference between values of the $\log \Gamma$ -function in close points, where Γ denotes Euler's gamma function. As a consequence, we obtain a way of computing the Dirichlet-multinomial log-likelihood function which is more accurate, has a better computational complexity and a wider range of application than the previously known ones.

Keywords Dirichlet multinomial distribution · Log-likelihood · Euler's Gamma

1 Introduction

It is well known that the Dirichlet-multinomial distribution plays a role in bayesian statistics, statistical inference, machine learning, data science; in magazine exposure, word burstiness and language modeling; in multiple sequence alignment and in counting data clustering. As an example, it is well known that a fundamental issue in data mining and machine learning is choosing the correct way to represent data. In the case of count data, when data suffer from overdispersion (i.e., their variance is larger than the statistical model would expect) one of the most commonly used way to model data is the Dirichlet-multinomial distribution, see Cameron and Trivedi (2013), Winkelmann (2008) and Agresti (2002). Furthermore, the Dirichlet-multinomial distribution is a very good candidate as a prior in the case of Bayesian belief nets, see Hooper (2004), and in the structural learning of graphical models, see Giudici and Castelo (2003). In the field of computational linguistic the

✉ Alessandro Languasco
alessandro.languasco@unipd.it

Mauro Migliardi
mauro.migliardi@unipd.it

¹ Dipartimento di Matematica, "Tullio Levi-Civita", Università di Padova, Via Trieste 63, 35121 Padua, Italy

² Dipartimento di Ingegneria dell'Informazione, Università di Padova, Via Gradenigo 6/b, 35131 Padua, Italy

Dirichlet-multinomial distribution is a well known tool and it is used in magazine-exposure modeling, see Leckenby and Kishi (1984), topic modeling, see Mimno and McCallum (2008), word-bursiness modeling, see Madsen et al. (2005), and, more in general, in language modeling, see MacKay and Bauman Peto (1994). In bioinformatics, the Dirichlet-multinomial distribution has been used to perform multiple sequence alignment, see Brown et al. (1993) and Sjölander et al. (1996), and, more recently, it has been applied in Bouguila (2008) to count data clusterings.

However, because of the shape of the $\log \Gamma$ -function (where Γ denotes Euler's gamma function) the known algorithms to compute the Dirichlet-multinomial log-likelihood function have some technical problems that limit their usability. In this paper we present a new algorithm to compute the difference between values of the $\log \Gamma$ -function in close points. The new algorithm allows us to obtain a way of computing the Dirichlet-multinomial log-likelihood function which is more accurate, has a better computational complexity and a wider range of application than the previously known ones.

In the following we will denote $\mathbb{N}$ as the set of natural numbers and $\mathbb{N}^*$ as $\mathbb{N} \setminus \{0\}$. Let $K, x_1, \dots, x_K \in \mathbb{N}^*$ and $N := \sum_{k=1}^K x_k$; let further $\alpha_1, \dots, \alpha_K > 0$, $A := \sum_{k=1}^K \alpha_k$, $\mathbf{x} = (x_1, \dots, x_K)$ and $\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_K)$.

The *Dirichlet-multinomial likelihood function* is defined as the following ratio of Euler's gamma function values

$$\mathcal{L}(\boldsymbol{\alpha}; \mathbf{x}) := \frac{\Gamma(A)}{\Gamma(A+N)} \prod_{k=1}^K \frac{\Gamma(\alpha_k + x_k)}{\Gamma(\alpha_k)}. \quad (1)$$

The log-likelihood function is then obtained by taking the logarithms of both sides of (1). Letting $\psi := 1/A$ be the *overdispersion* parameter and $p_1, \dots, p_K > 0$ such that $\sum_{k=1}^K p_j = 1$, we also set $\alpha_k = p_k/\psi$, $k = 1, \dots, K$. So, rewriting the previous formula by letting $\mathbf{p} := (p_1, \dots, p_K)$, we denote the *Dirichlet-multinomial log-likelihood* function as

$$\begin{aligned} \log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x}) := & -\log \Gamma\left(\frac{1}{\psi} + N\right) + \log \Gamma\left(\frac{1}{\psi}\right) \\ & + \sum_{k=1}^K \left(\log \Gamma\left(\frac{p_k}{\psi} + x_k\right) - \log \Gamma\left(\frac{p_k}{\psi}\right) \right). \end{aligned} \quad (2)$$

The standard way of evaluating $\log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x})$ is to compute first $\mathcal{L}(\boldsymbol{\alpha}, \mathbf{x})$, where $\boldsymbol{\alpha} = \mathbf{p}/\psi$ and $\mathbf{x}$ is as defined in (1). This is the way the VGAM package of the R system works, see Yee (2010, 2022) and Yee and Wild (1996). Using the functional equation $\Gamma(u+1) = u\Gamma(u)$ for $u > 0$, the computation of $\mathcal{L}(\boldsymbol{\alpha}; \mathbf{x})$ is reduced to evaluate the ratio of two products of T real numbers, see Yee (2022, pp. 238–240), where T is proportional to N ; but, when N or the involved numbers are large (as in the overdispersed case), such products may become huge. This leads to have overflow problems or to a very long execution time. To overcome this problem one might write $\log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x})$ as a sum of T logarithms, where T is proportional to N , see for example (15) below. This is the methodology (Bouneffouf 2020) proposed in his preprint.

In this case the computation requires to evaluate T times the logarithm function; a task that becomes costly from a computational point of view whenever N is large. More remarks on this methodology are contained in Remark 1.

Moreover, the applications in which ψ tends to 0 are difficult to handle since in this case each $\log \Gamma$ -term becomes huge, but the paired differences in (2) are still relatively small. Because of the limited accuracy of the floating point arithmetic, the large terms cancel and the result is ridden with large errors.

To avoid this problem, Yu and Shaw (2014) found a different method to evaluate $\log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x})$ which is stable for $\psi \rightarrow 0^+$ and faster than the previous mentioned ones; they also showed its applicability in the field of human microbiome data counting. Letting $x > 0$, $y \in \mathbb{N}^*$, $x(y-1) < 1$, they stated that

$$\log \Gamma\left(\frac{1}{x} + y\right) - \log \Gamma\left(\frac{1}{x}\right) = -y \log x + \sum_{\ell=2}^{\infty} \frac{(-1)^{\ell} \phi_{\ell}(y)}{\ell(\ell-1)} x^{\ell-1}, \quad (3)$$

where, thanks to Faulhaber's formula, we have

$$\phi_n(m) := n \sum_{k=0}^{m-1} k^{n-1} = B_n(m) - B_n(0), \quad (4)$$

$m \in \mathbb{N}^*$, and $B_n(u)$ are the *Bernoulli polynomials* of order $n \geq 2$. The polynomials $\phi_n(u)$ are also called the *old-Bernoulli polynomials* of order $n \geq 2$. We remark that the condition $x(y-1) < 1$, is needed to ensure the absolute convergence of the series in (3).

In fact, Yu and Shaw used twice the asymptotic formula for $\log \Gamma(z+a)$, $z, a \in \mathbb{C}$, $|z| \rightarrow \infty$, first proved by Barnes (1900), but already studied by Hermite (1895), to sketch a proof of (3).

Unfortunately, while developing their methodology, see Eq. (8)–(12) of Yu and Shaw (2014), the authors stopped taking into account the fact that the formula they are using is an asymptotic one and that, for this reason, it can be used only when z is sufficiently large; this misuse is also proven by the fact that they removed the asymptotic error term $O(|z|^{-m})$ from their result. In the experiments described in Yu and Shaw (2014), they never verified if z is large enough to ensure that Barnes formula holds and they never considered the presence of the asymptotic error term $O(|z|^{-m})$.

Nonetheless, the correctness of (3), and, as a consequence, the fact that their experiments provided reliable results, can be proved. We will provide the correct proof in Sect. 2 by showing that we can obtain (3) just exploiting the Taylor formula for $\log(1+u)$. This will also allow us to evaluate the error term one obtains in (3) by replacing the series with a finite sum, showing that this leads to an error-adaptive algorithm, a feature Yu and Shaw did not explore. Finally, our approach also allowed us to correct and improve their error term estimate.

In practical applications, they also had to overcome the condition $x(y-1) < 1$; their solution was to introduce a set of intermediate points—a *mesh* of points—to split the computation of $\log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x})$ in suitable subintervals, see Section 4 of Yu and Shaw (2014).

In this work we will improve on Yu–Shaw’s algorithm with regards to both the uniformity in x, y and the computational complexity. For $x \geq 0, y \in \mathbb{N}^*, m \in \mathbb{N}^*$, we define

$$d(x, y) := (1 + x(y - 1))^{-1}, \quad (5)$$

$$S_{2m}(x, y) := \sum_{n=1}^m \frac{B_{2n} x^{2n-1}}{2n(2n-1)} (d(x, y)^{2n-1} - 1), \quad (6)$$

where B_{2n} are the even-index Bernoulli numbers; we recall that, with the unique exception of $B_1 = -1/2$, the odd-index Bernoulli numbers are equal to 0. Moreover, for $x > 0$, we define

$$E_{2m}(x, y) := \frac{1}{2m} \int_{1/x}^{1/x+y-1} B_{2m} \left(\left\{ t - \frac{1}{x} \right\} \right) \frac{dt}{t^{2m}}, \quad (7)$$

where we denoted as $\{u\}$ the fractional part of $u \in \mathbb{R}$ and $B_{2n}(u)$ are the even-index Bernoulli polynomials. We remark that $S_{2m}(x, 1) = 0$ and $E_{2m}(x, 1) = 0$. We also define $E_{2m}(0, y) := 0$ for every $y \in \mathbb{N}^*$. We are now ready to state the following

Theorem 1 Let $x > 0, y \in \mathbb{N}, y \geq 3, m \in \mathbb{N}^*$. Then

$$\log \Gamma \left(\frac{1}{x} + y \right) - \log \Gamma \left(\frac{1}{x} \right) = -y \log x + 1 - y - \left(\frac{1}{x} + y - \frac{1}{2} \right) \log d(x, y) + S_{2m}(x, y) + E_{2m}(x, y), \quad (8)$$

where Γ is Euler’s gamma function, $d(x, y)$, $S_{2m}(x, y)$ and $E_{2m}(x, y)$ are respectively defined in (5)–(7). We further have that

$$|E_{2m}(x, y)| < 2 \left(1 + \frac{1}{4^{m+1}} \frac{2m+3}{2m+1} \right) \frac{x^{2m+1} (2m)!}{(2\pi)^{2m+2}} (1 - d(x, y)^{2m+1}). \quad (9)$$

Moreover, for $y \in \{1, 2\}$ we have

$$\log \Gamma \left(\frac{1}{x} + y \right) - \log \Gamma \left(\frac{1}{x} \right) = -y \log x - (y - 1) \log d(x, y). \quad (10)$$

The proof of Theorem 1, presented in Sect. 3, makes use of the Euler–Maclaurin summation formula whose statement is given in Lemma 1. We also remark that for $m, y \in \mathbb{N}^*$ both fixed, the function $1 - y - (1/x + y - 1/2) \log d(x, y) + S_{2m}(x, y)$ tends to 0 as $x \rightarrow 0^+$ thus showing that $\log \Gamma(1/x + y) - \log \Gamma(1/x) + y \log x$ can be prolonged to an analytic function in $x = 0$ for every $y \in \mathbb{N}^*$.

A well known characteristic of the Euler–Maclaurin summation formula, see, e.g., Stoer and Bulirsch (2002), Sections 3.3–3.4, is that, for $x, y > 0$ fixed, $E_{2m}(x, y)$ might not converge to 0 as $m \rightarrow \infty$. In particular, it is clear that for $x \geq 1$, $E_{2m}(x, y)$, defined in (7), diverges as $m \rightarrow \infty$. But, for $x < 1$, it is still possible to use Theorem 1 to efficiently and accurately evaluate $\log \Gamma(1/x + y) - \log \Gamma(1/x)$, since

another well known characteristic of $E_{2m}(x, y)$, $x < 1$ and $y \in \mathbb{N}^*$ both fixed, is that it decreases for the first values of m until it reaches a minimal value for some $\tilde{m}$. Since in all the applications we will have $x < 1$ (in fact, x will play the role of ψ and of ψ/p_k) in (2), we can first determine $\tilde{m}$ and evaluate $|E_{2\tilde{m}}(x, y)|$; if it is sufficiently small for our goals we then proceed to evaluate the remaining part of (8). Moreover, it is easy to see that the order of magnitude of the right hand side of (9) has a weak dependence from large values of y ; hence, for $x < 1$ fixed, we can find $\tilde{m}$ such that

$$E_{2m}(x) := \left(1 + \frac{1}{4^{m+1}} \frac{2m+3}{2m+1}\right) \frac{2(2m)!}{(2\pi)^{2m+2}} x^{2m+1} \quad (11)$$

is minimal and then use $S_{2\tilde{m}}(x, y)$ and $E_{2\tilde{m}}(x, y)$ in (8) for every $y \geq 3$, since $|E_{2\tilde{m}}(x, y)| \leq E_{2\tilde{m}}(x)$. We further remark that the coefficients in (11) can be precomputed. In practice, this procedure for $0 < x < 1$ nicely works and the smaller x is, the smaller $E_{2\tilde{m}}(x)$ becomes; this is the case in which several applications work. In Sect. 4 we will compare (11) with the corrected error term in Yu-Shaw's result which, we recall, works only in the restricted range $x(y-1) < 1$.

In the case $x \geq 1$, or even if $x < 1$ but $E_{2\tilde{m}}(x)$ is not small enough for our goals, we can rearrange the computation as follows. Letting $h \in \mathbb{N}$, $3 \leq h \leq y$, and $1/v := 1/x + h$, we have $v = x/(1+hx) < 1/h \leq 1/3$. Moreover, letting $z = y - h \in \mathbb{N}$, we can write that $\log \Gamma(1/x + y) = \log \Gamma(1/v + z)$ and hence

$$\begin{aligned} \log \Gamma\left(\frac{1}{x} + y\right) - \log \Gamma\left(\frac{1}{x}\right) &= \log \Gamma\left(\frac{1}{v} + z\right) - \log \Gamma\left(\frac{1}{v}\right) + \log \Gamma\left(\frac{1}{v}\right) - \log \Gamma\left(\frac{1}{x}\right) \\ &= \log \Gamma\left(\frac{1}{v} + z\right) - \log \Gamma\left(\frac{1}{v}\right) + \sum_{j=0}^{h-1} \log\left(\frac{1}{x} + j\right), \end{aligned} \quad (12)$$

in which we used the functional equation $\Gamma(u+1) = u\Gamma(u)$ for $u > 0$. In the following, we will refer to this as the *horizontal shift* idea. Now we can apply Theorem 1 to $\log \Gamma(1/v + z) - \log \Gamma(1/v)$ in a situation, $v < 1/h \leq 1/3$, in which we have a better accuracy of the final result. Hence, by choosing a sufficiently large h , and at the cost of evaluating h logarithms (or a logarithm of a product having h factors; a quantity that might be evaluated more efficiently if x is not too small), we can obtain a much smaller estimate for $E_{2m}(v)$ than the one available for $E_{2m}(x)$. In Table 1 below, we insert an example of error computation for $E_{2m}(1/15)$, which essentially corresponds to choosing $h = 15$ in the previous discussion; since $E_{2\tilde{m}}(1/15) \approx 9.62651 \times 10^{-43}$, $\tilde{m} = 47$, with this h -choice we have an accuracy of at least 40 decimal digits for every possible choice of $y \in \mathbb{N}$, $y \geq 3$.

Recalling that $K, x_1, \dots, x_K \in \mathbb{N}^*$, $N = \sum_{k=1}^K x_k$, $\psi > 0$ and $p_1, \dots, p_K > 0$ such that $\sum_{k=1}^K p_j = 1$, for every $m \in \mathbb{N}^*$ we define

Table 1 Truncated estimates for the error terms for $x = 2 \times 10^{-4}$, $y = 10^3$ (on the first part) and for $x = 1/15$, $y = 3$ (on the second part)

m	$f(x, y, m)$	$g(x, y, m)$
1	100.00000	9.54452×10^{-15}
2	6.66666	1.52495×10^{-22}
3	0.66666	5.49539×10^{-30}
4	0.08000	3.47393×10^{-37}
5	0.01066	3.39772×10^{-44}
6	0.00152	4.75945×10^{-51}
7	0.00022	9.05311×10^{-58}
8	3.55555×10^{-5}	2.24820×10^{-64}
9	5.68888×10^{-6}	7.07112×10^{-71}
10	9.30909×10^{-7}	2.74947×10^{-77}
11	1.55151×10^{-7}	1.29580×10^{-83}
12	2.62564×10^{-8}	7.28137×10^{-90}
13	4.50109×10^{-9}	4.81099×10^{-96}
14	7.80190×10^{-10}	3.69345×10^{-102}
15	1.36533×10^{-10}	3.26083×10^{-108}
16	2.40941×10^{-11}	3.28102×10^{-114}
17	4.28339×10^{-12}	3.73275×10^{-120}
18	7.66502×10^{-13}	4.76788×10^{-126}
19	1.37970×10^{-13}	6.79467×10^{-132}
20	2.49660×10^{-14}	1.07424×10^{-137}
m	$f(x, y, m)$	$g(x, y, m)$
1	0.30000	2.62851×10^{-7}
2	0.02000	4.88337×10^{-10}
3	0.00200	2.03510×10^{-12}
4	0.00024	1.48006×10^{-14}
5	3.20000×10^{-5}	1.65742×10^{-16}
6	4.57142×10^{-6}	2.64656×10^{-18}
7	6.85714×10^{-7}	5.71600×10^{-20}
8	1.06666×10^{-7}	1.60615×10^{-21}
9	1.70666×10^{-8}	5.69873×10^{-23}
10	2.79272×10^{-9}	2.49309×10^{-24}
11	4.65454×10^{-10}	1.31904×10^{-25}
12	7.87692×10^{-11}	8.30511×10^{-27}
13	1.35032×10^{-11}	6.13896×10^{-28}
14	2.34057×10^{-12}	5.26567×10^{-29}
15	4.09600×10^{-13}	5.18851×10^{-30}
16	7.22823×10^{-14}	5.82151×10^{-31}

On the fast computation of the Dirichlet-multinomial...

Table 1 (continued)

m	$f(x, y, m)$	$g(x, y, m)$
17	1.28501×10^{-14}	7.37997×10^{-32}
18	2.29950×10^{-15}	1.04978×10^{-32}
19	4.13911×10^{-16}	1.66528×10^{-33}
20	7.48982×10^{-17}	2.92958×10^{-34}

$f(x, y, m)$ is the right hand side of (21) and $g(x, y, m)$ is the right hand side of (9)

$$\begin{aligned} \mathcal{M}(\mathbf{p}, \psi; \mathbf{x}, m) &:= K - 1 + \left(\frac{1}{\psi} + N - \frac{1}{2} \right) \log d(\psi, N) - S_{2m}(\psi, N) \\ &+ \sum_{k=1}^K \left[x_k \log p_k - \left(\frac{p_k}{\psi} + x_k - \frac{1}{2} \right) \log d\left(\frac{\psi}{p_k}, x_k \right) + S_{2m}\left(\frac{\psi}{p_k}, x_k \right) \right] \end{aligned} \quad (13)$$

and

$$\mathcal{E}(\mathbf{p}, \psi; \mathbf{x}, m) := -E_{2m}(\psi, N) + \sum_{k=1}^K E_{2m}\left(\frac{\psi}{p_k}, x_k \right). \quad (14)$$

We remark that (13)–(14) hold for $\psi = 0$ too by letting $\mathcal{M}(\mathbf{p}, 0; \mathbf{x}, m) := \sum_{k=1}^K x_k \log p_k$ and $\mathcal{E}(\mathbf{p}, 0; \mathbf{x}, m) := 0$ for every $\mathbf{p}, \mathbf{x}$ and $m \in \mathbb{N}^*$. Moreover, for every $\mathbf{p}, \mathbf{x}$ and $m \in \mathbb{N}^*$, $\mathcal{M}(\mathbf{p}, \psi; \mathbf{x}, m)$ and $\mathcal{E}(\mathbf{p}, \psi; \mathbf{x}, m)$ can be prolonged to analytic functions at $\psi = 0$.

Combining the definitions of $\mathbf{p}$ and $\mathbf{x}$, Eq. (2) and Theorem 1, we can write $\log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x})$ in the following way.

Corollary 1 Let $K, x_1, \dots, x_K \in \mathbb{N}^*$ and $N = \sum_{k=1}^K x_k$. Let further $p_1, \dots, p_K > 0$ such that $\sum_{k=1}^K p_j = 1$ and $\alpha_k = p_k/\psi$, where $\psi > 0$ is fixed. Moreover, denote $\mathbf{x} = (x_1, \dots, x_K)$, $\mathbf{p} = (p_1, \dots, p_K)$ and let $\log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x})$ be defined as in (2). For every $m \in \mathbb{N}^*$, we have

$$\log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x}) = \mathcal{M}(\mathbf{p}, \psi; \mathbf{x}, m) + \mathcal{E}(\mathbf{p}, \psi; \mathbf{x}, m),$$

where $\mathcal{M}(\mathbf{p}, \psi; \mathbf{x}, m)$ and $\mathcal{E}(\mathbf{p}, \psi; \mathbf{x}, m)$ are defined in (13)–(14).

Using the previous remarks and exploiting the well known asymptotic formula $\log(1+u) \sim u$ as $u \rightarrow 0^+$, we have $\lim_{\psi \rightarrow 0^+} \log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x}) = \sum_{k=1}^K x_k \log p_k$ and this shows the stability of our result.

Corollary 1 also shows an *error accumulation* phenomenon in computing $\log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x})$; to be more precise, let $\Delta \in (0, 1)$ be fixed and assume that there exists $\bar{m} \in \mathbb{N}^*$, such that $|E_{\bar{m}}(\psi, N)| < \Delta$ and $|E_{\bar{m}}(\psi/p_k, x_k)| < \Delta$ for every $k = 1, \dots, K$. Then $|\mathcal{E}(\mathbf{p}, \psi; \mathbf{x}, \bar{m})| < (K+1)\Delta$. A similar phenomenon occurs also in Eq. (20) of Yu and Shaw (2014) but in their case the situation is worse since, for being sure that the condition $x(y-1) < 1$ holds, and hence to be able to use a truncated version of

(3), they have to further split the sum over k in (2) in L subsums, where $L > K$ is the number of points of the mesh; thus essentially summing for $(L + 1)$ -times an error Δ obtained in one approximated evaluation of (3). Unfortunately, Yu and Shaw did not mention this phenomenon in their work.

We finally remark that in several applications the goal is to find the extremal values for $\log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x})$ or for very similar quantities. Starting from $\log \Gamma(u + y) - \log \Gamma(u)$, $u > 0$, $y \in \mathbb{N}^*$, and differentiating with respect to u , a relevant quantity is $f(u + y) - f(u)$, where $f(u) := \Gamma'(u)/\Gamma(u)$, $u > 0$, is the *digamma*¹ function and $\Gamma'(u)$ denotes the first derivative of the gamma function. Using the relation $f(u + 1) = f(u) + 1/u$ for $u > 0$, which immediately follows from the functional equation $\Gamma(u + 1) = u\Gamma(u)$, to look for the extremal values of $\log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x})$ we need to evaluate

$$f(\mathbf{p}, \psi; \mathbf{x}) := f\left(\frac{1}{\psi}\right) - f\left(\frac{1}{\psi} + N\right) + \sum_{k=1}^K \left(f\left(\frac{p_k}{\psi} + x_k\right) - f\left(\frac{p_k}{\psi}\right) \right)$$

in which the key quantity is

$$f\left(\frac{1}{x} + y\right) - f\left(\frac{1}{x}\right) = \sum_{j=0}^{y-1} \frac{1}{1/x + j} = x \left(1 + \sum_{j=1}^{y-1} \frac{1}{1 + jx} \right)$$

for $x > 0$, $y \in \mathbb{N}^*$. We remark that the quantity at the right hand side of the last formula is defined for $x = 0$ too. To compute $\sum_{j=0}^{y-1} (1/x + j)^{-1}$ we can use the same methodology we adopted in the proof of Theorem 1 thus getting the following

Theorem 2 Let $x > 0$, $y \in \mathbb{N}$, $y \geq 3$, $m \in \mathbb{N}^*$. Then

$$f\left(\frac{1}{x} + y\right) - f\left(\frac{1}{x}\right) = -\log d(x, y) + \frac{x}{2}(1 + d(x, y)) + \sum_{n=1}^m \frac{B_{2n}}{2n} x^{2n} (1 - d(x, y)^{2n}) - \int_{1/x}^{1/x+y-1} B_{2m}\left(\left\{t - \frac{1}{x}\right\}\right) \frac{dt}{t^{2m+1}},$$

where f is the digamma function, B_{2n} are the even-index Bernoulli numbers, $B_{2n}(u)$ are the even-index Bernoulli polynomials and $d(x, y)$ is defined in (5). We further have that

$$\left| \int_{1/x}^{1/x+y-1} B_{2m}\left(\left\{t - \frac{1}{x}\right\}\right) \frac{dt}{t^{2m+1}} \right| < \left(1 + \frac{1}{4^{m+1}} \frac{2m+3}{2m+1} \right) \frac{2(2m+1)!}{(2\pi)^{2m+2}} (1 - d(x, y)^{2m+2}) x^{2m+2}.$$

Moreover, we have

¹ In the literature the digamma and polygamma functions are respectively denoted as $\psi(u)$, $\psi^{(\ell)}(u)$ but in this context ψ has a different meaning.

$$F\left(\frac{1}{x} + 1\right) - F\left(\frac{1}{x}\right) = x \quad \text{and} \quad F\left(\frac{1}{x} + 2\right) - F\left(\frac{1}{x}\right) = x(1 + d(x, 2)).$$

Analogous results can be obtained for the *polygamma*² functions, i.e., the higher order derivatives of $\log \Gamma$, since $F^{(\ell)}(u + 1) = F^{(\ell)}(u) + (-1)^\ell (\ell!)/u^{\ell+1}$, for every $\ell \in \mathbb{N}^*$ and $u > 0$. The relevant function in this case is

$$F^{(\ell)}(\mathbf{p}, \boldsymbol{\psi}; \mathbf{x}) := F^{(\ell)}\left(\frac{1}{\boldsymbol{\psi}}\right) - F^{(\ell)}\left(\frac{1}{\boldsymbol{\psi}} + N\right) + \sum_{k=1}^K \left(F^{(\ell)}\left(\frac{p_k}{\boldsymbol{\psi}} + x_k\right) - F^{(\ell)}\left(\frac{p_k}{\boldsymbol{\psi}}\right) \right)$$

in which the key quantity is

$$F^{(\ell)}\left(\frac{1}{x} + y\right) - F^{(\ell)}\left(\frac{1}{x}\right) = (-1)^\ell (\ell!) \sum_{j=0}^{y-1} \frac{1}{(1/x + j)^{\ell+1}}$$

for $x > 0$, $y \in \mathbb{N}^*$. We obtain, again arguing as in the proof of Theorem 1, the following

Theorem 3 Let $x > 0$, $y \in \mathbb{N}$, $y \geq 3$, $m, \ell \in \mathbb{N}^*$. Then

$$\begin{aligned} F^{(\ell)}\left(\frac{1}{x} + y\right) - F^{(\ell)}\left(\frac{1}{x}\right) &= (-x)^\ell (\ell!) \left[\frac{1 - d(x, y)^\ell}{\ell} + \frac{x}{2} (1 + d(x, y)^{\ell+1}) \right. \\ &\quad \left. + \sum_{n=1}^m \frac{B_{2n}(2n + \ell - 1)!}{(\ell!)(2n)!} x^{2n} (1 - d(x, y)^{2n+\ell}) \right] \\ &\quad + (-1)^{\ell+1} \frac{(2m + \ell)!}{(2m)!} \int_{1/x}^{1/x+y-1} B_{2m}\left(t - \frac{1}{x}\right) \frac{dt}{t^{2m+\ell+1}}, \end{aligned}$$

where $F^{(\ell)}$ is the ℓ -th polygamma function, B_{2n} are the even-index Bernoulli numbers, $B_{2n}(u)$ are the even-index Bernoulli polynomials and $d(x, y)$ is defined in (5). We further have that

$$\begin{aligned} &\left| \frac{(2m + \ell)!}{(2m)!} \int_{1/x}^{1/x+y-1} B_{2m}\left(t - \frac{1}{x}\right) \frac{dt}{t^{2m+\ell+1}} \right| \\ &< \left(1 + \frac{1}{4^{m+1}} \frac{2m + 3}{2m + 1} \right) \frac{2(\ell + 2m + 1)!}{(2\pi)^{2m+2}} (1 - d(x, y)^{2m+\ell+2}) x^{2m+\ell+2}. \end{aligned}$$

Moreover, for $\ell \in \mathbb{N}^*$ we have

$$F^{(\ell)}\left(\frac{1}{x} + 1\right) - F^{(\ell)}\left(\frac{1}{x}\right) = (-1)^\ell (\ell!) x^{\ell+1},$$

and

$$F^{(\ell)}\left(\frac{1}{x} + 2\right) - F^{(\ell)}\left(\frac{1}{x}\right) = (-1)^\ell (\ell!) x^{\ell+1} (1 + d(x, 2)^{\ell+1}).$$

² See the previous footnote.

We finally remark that the horizontal shift idea presented in (12) can be suitably adapted to be used for the digamma and polygamma functions too.

1.1 Outline

The paper is organised as follows. Sect. 2 is dedicated to prove (3) and to obtain a sharper error term estimate that leads to an error-adaptive version of Yu–Shaw’s mesh algorithm. In Sect. 3 we will prove Theorem 1. In Sect. 4 we will compare the accuracy and computational complexity of the algorithm that uses the formulae in Theorem 1 with the ones of the mesh algorithm. Finally, in Sect. 5, we replicate some of the experiments described in Yu–Shaw’s work to show that our algorithm is much faster than theirs.

2 Improved Yu–Shaw’s result

Here we give a proof of (3) and we will also improve the error term estimate in Yu and Shaw (2014). Let $x > 0$, $y \in \mathbb{N}^*$. From the functional equation of Euler’s gamma function, $\Gamma(u + 1) = u\Gamma(u)$ for $u > 0$, we immediately obtain

$$\log \Gamma\left(\frac{1}{x} + y\right) - \log \Gamma\left(\frac{1}{x}\right) = \sum_{j=0}^{y-1} \log\left(\frac{1}{x} + j\right) = -y \log x + \sum_{j=1}^{y-1} \log(1 + jx). \quad (15)$$

Inserting the Taylor formula of $\log(1 + u) = \sum_{n=0}^{\infty} (-1)^n u^{n+1} / (n + 1)$, $u \in (-1, 1]$, we get

$$\log \Gamma\left(\frac{1}{x} + y\right) - \log \Gamma\left(\frac{1}{x}\right) = -y \log x + \sum_{n=0}^{\infty} \frac{(-1)^n x^{n+1}}{n + 1} \sum_{j=1}^{y-1} j^{n+1} \quad (16)$$

and the series on the right hand side absolutely converges for $x(y - 1) < 1$. Recalling (4) and letting $\ell = n + 2$, we immediately obtain (3). Let now $m \in \mathbb{N}$, $m \geq 2$,

$$D_m(x, y) := \sum_{\ell=2}^m \frac{(-1)^\ell x^{\ell-1} \phi_\ell(y)}{\ell(\ell - 1)} = \sum_{n=0}^{m-2} \frac{(-1)^n x^{n+1}}{n + 1} \sum_{j=1}^{y-1} j^{n+1}, \quad (17)$$

and $D_1(x, y) = D_2(x, y) := 0$. Exploiting that t^s , $t > 0$, $s > 1$, is an increasing function, we remark that

$$\phi_\ell(y) = \ell \sum_{j=1}^{y-1} j^{\ell-1} \leq \ell \int_1^y t^{\ell-1} dt < y^\ell \quad (18)$$

for $\ell \geq 1$. Following the argument in (Yu and Shaw 2014, Section 3), to estimate the error in substituting the series in (3), or in (16), with $D_m(x, y)$, for $x > 0$, $y \in \mathbb{N}^*$ and $x(y - 1) < 1$, we obtain, using (17)–(18), that

$$\left| \sum_{\ell=2}^{\infty} \frac{(-1)^{\ell} x^{\ell-1} \phi_{\ell}(y)}{\ell(\ell-1)} - D_m(x, y) \right| < y \sum_{\ell=m+1}^{\infty} \frac{(xy)^{\ell-1}}{\ell(\ell-1)} < \frac{(xy)^m y}{m(m+1)(1-xy)}, \quad (19)$$

in which we also used the formula for the geometric series of ratio $xy \in (0, 1)$. We remark that, comparing Eq. (15) of Yu and Shaw (2014) with (19), it seems that an extra factor ℓx is included there thus getting a stronger, but wrong, estimate for the error term; see also the remarks after the end of this proof.

Let now $\Delta \in (0, 1)$ be fixed. For $x > 0$, $y \in \mathbb{N}$, $y \geq 2$, $x(y-1) < 1$, to obtain that the right hand side of (19) is less than Δ , it is enough to choose

$$m > \frac{|\log \Delta| + |\log(1-xy)| + \log y}{|\log(xy)|} \quad (20)$$

thus showing that the optimal choice of m depends from the desired accuracy Δ . In this way we proved (3), and we also fixed a mistake in Yu–Shaw error term estimate (Eq. (15) of Yu and Shaw (2014)).

We finally remark that (19) can be improved using the error term estimate for the alternating series of $\log(1+jx)$. In fact, starting from (15)–(17) and again assuming $m \in \mathbb{N}$, $m \geq 2$, $x > 0$, $y \in \mathbb{N}^*$ and $x(y-1) < 1$, the error term, using also (18), is

$$\left| \sum_{j=1}^{y-1} \log(1+jx) - \sum_{j=1}^{y-1} \sum_{n=0}^{m-2} \frac{(-1)^n (jx)^{n+1}}{n+1} \right| \leq \sum_{j=1}^{y-1} \left| \sum_{n=m-1}^{\infty} \frac{(-1)^n (jx)^{n+1}}{n+1} \right| < \frac{(xy)^m y}{m(m+1)} \quad (21)$$

which is sharper than (19). As a consequence, (20) is improved too and becomes $m > \frac{|\log \Delta| + \log y}{|\log(xy)|}$. $\square$

We have two further remarks on (19) and the corresponding Eq. (15) of Yu and Shaw (2014):

- (1) in Yu and Shaw (2014) the authors claim to use $\Delta \approx 1.3 \times 10^{-16}$ by choosing $m = 20$ uniformly for $xy \leq \delta = 1/5$; but the missing factor ℓx we mentioned before in Eq. (15) of Yu and Shaw (2014) affected their estimate. For example, using (19) with $x = 2 \times 10^{-4}$, for $m = 20$ we have $\Delta \approx 3.1207 \times 10^{-14}$; their erroneous claim $\Delta \approx 1.3 \times 10^{-16}$ would follow by multiplying 3.1207×10^{-14} by the missing factor $(m+1)x = 42 \times 10^{-4}$;
- (2) from (20) it follows that the choice of m depends on x, y, Δ ; this means that an algorithm based on (16) and (19) has an adaptive error term estimate but in Yu and Shaw (2014) the authors fix a very small error Δ they use uniformly for every x, y such that $x(y-1) < 1$; this means that they use more summands than needed to obtain a result having a Δ -accuracy.

3 Proof of Theorem 1

Let $x > 0$, $y \in \mathbb{N}^*$. The starting point is formula (15) but now we use the Euler–Maclaurin summation formula on $\sum_{j=0}^{y-1} \log(1/x + j)$. We will need the following lemma whose proof follows from combining the topics in Sections 3.3–3.4 of Stoer and Bulirsch (2002).

Lemma 1 (Euler–Maclaurin formula) *Let $a \in \mathbb{R}$, $N \in \mathbb{N}$, and $m \in \mathbb{N}^*$. Assume that $f \in C^{2m+4}([a, N+a])$ and that both $f^{(2m+2)}(t), f^{(2m+4)}(t)$ have constant sign on $[a, a+N]$. We have*

$$\begin{aligned} \sum_{j=0}^N f(j+a) &= \int_a^{N+a} f(t) dt + \frac{1}{2} (f(N+a) + f(a)) \\ &\quad + \sum_{n=1}^m \frac{B_{2n}}{(2n)!} (f^{(2n-1)}(N+a) - f^{(2n-1)}(a)) \\ &\quad - \frac{1}{(2m)!} \int_a^{N+a} B_{2m}(\{t-a\}) f^{(2m)}(t) dt, \end{aligned}$$

where $B_n(u)$ are the Bernoulli polynomials and B_n are the Bernoulli numbers of order n . Moreover

$$\left| \frac{1}{(2m)!} \int_a^{N+a} B_{2m}(\{t-a\}) f^{(2m)}(t) dt \right| \leq \frac{|B_{2m+2}|}{(2m+2)!} |f^{(2m+1)}(N+a) - f^{(2m+1)}(a)|.$$

Recalling, by (15), that

$$\log \Gamma\left(\frac{1}{x} + y\right) - \log \Gamma\left(\frac{1}{x}\right) = \sum_{j=0}^{y-1} \log\left(\frac{1}{x} + j\right), \quad (22)$$

we apply the first part of Lemma 1 to $f(t) = \log t$, $a = 1/x$ and $N = y-1$ thus obtaining that

$$\begin{aligned} \sum_{j=0}^{y-1} \log\left(\frac{1}{x} + j\right) &= \int_{1/x}^{1/x+y-1} \log t dt + \frac{1}{2} \left(\log\left(\frac{1}{x} + y-1\right) + \log \frac{1}{x} \right) \\ &\quad + \sum_{n=1}^m \frac{B_{2n} x^{2n-1}}{2n(2n-1)} (d(x, y)^{2n-1} - 1) \\ &\quad - \frac{1}{2m} \int_{1/x}^{1/x+y-1} B_{2m}\left(\left\{t - \frac{1}{x}\right\}\right) \frac{dt}{t^{2m}}, \end{aligned} \quad (23)$$

since $(\log t)^{(\beta)} = (-1)^{\beta-1} t^{-\beta} (\beta-1)!$ for every $t > 0$ and $\beta \in \mathbb{N}^*$; in the previous computation we also made use of (5). Combining (22)–(23), recalling that $\int_v^u \log t dt = u \log u - u - v \log v + v$ and performing the needed computations in

On the fast computation of the Dirichlet-multinomial...

(23), the first part of Theorem 1 follows. Now we prove the second part of Theorem 1. Using the second part of Lemma 1, we also have

$$\left| \frac{1}{2m} \int_{1/x}^{1/(x+y-1)} B_{2m} \left(\left\{ t - \frac{1}{x} \right\} \right) \frac{dt}{t^{2m}} \right| \leq \frac{|B_{2m+2}| (2m)! x^{2m+1}}{(2m+2)!} |d(x, y)^{2m+1} - 1|$$

$$< \left(1 + \frac{1}{4^{m+1}} \frac{2m+3}{2m+1} \right) \frac{2(2m)!}{(2\pi)^{2m+2}} x^{2m+1} (1 - d(x, y)^{2m+1})$$

in which we used the formula $B_{2m} = 2(-1)^{m-1} (2\pi)^{-2m} \zeta(2m) (2m)!$, where $m \in \mathbb{N}^*$, $\zeta(s)$, $s > 1$, denotes the Riemann zeta-function, and the estimate

$$\zeta(2m) = 1 + \frac{1}{4^m} + \sum_{k=3}^{\infty} \frac{1}{k^{2m}} < 1 + \frac{1}{4^m} + \int_2^{\infty} \frac{dt}{t^{2m}} = 1 + \frac{1}{4^m} \frac{2m+1}{2m-1},$$

which follows since t^{-2m} is a decreasing function. This proves the second part of Theorem 1. $\square$

4 Comparing error terms and computational complexities

We now compare the error terms and the computational complexities of (3) and of the formula in Theorem 1.

4.1 Comparing error terms

In the applications, in fact, we will always have $y \geq 3$ otherwise we can directly use (10). Assuming that the logarithm functions can be evaluated with the same accuracy both in (3) and in (8), the key point is that the estimate in (9) uniformly holds for every $0 < x < 1$, $y \geq 3$, instead of $0 < x(y-1) < 1$. This implies that, for being able to perform the error-terms comparison, in what follows we have to assume $0 < x \leq 1/3$. This also means that we can get rid of the necessity of building a mesh of points.

Let $f(x, y, m)$ be the function on the right hand side of (21) and $g(x, y, m)$ be the one on the right hand side of (9). We first choose $x = 2 \times 10^{-4}$, $y = 10^3$, so that both (19) and (21) are applicable, and $xy = 1/5$, which is compatible with the setting in Yu and Shaw (2014). Then we consider the worst possible case $x = 1/15$, $y = 3$ ($xy = 1/5$) for $g(x, y, m)$, see the discussion Sect. 1 about the error term in Theorem 1; the optimal $\tilde{m}$ in this case is 47 and $E_{94}(1/15) \approx 9.62651 \times 10^{-43}$.

To show that $g(x, y, m)$ is much smaller than $f(x, y, m)$ in the previously mentioned cases, we wrote a short program using the scripting language of Pari/GP (2022); we decided to use Pari/GP (a CAS developed by the University of Bordeaux to provide computational tools in Number Theory) to perform this task since it allows an easy access to a multiprecision arithmetic and many of the functions here mentioned are already implemented there. A quick computation using Pari/GP gives Table 1 in which it is clear that the estimate obtained with $g(x, y, m)$ is

much stronger than the one which uses $f(x, y, m)$ for each $1 \leq m \leq 20$. Letting now $\Delta \in (0, 1)$ be fixed, this implies that we need a much smaller value of m to achieve a Δ -accuracy result. Moreover $g(x, y, m)$ can be used for every $x < 1$ while $f(x, y, m)$ needs that $x(y - 1) < 1$. This means that the mesh step is not needed at all and it is clear the huge gain in speed and accuracy we can obtain using Theorem 1, or Corollary 1, comparing with the algorithm in Yu and Shaw (2014).

4.2 Comparing computational complexities

From a computational point of view, the main difference between (3) and (8) is that we replaced the degree ℓ old-Bernoulli polynomial $\phi_\ell(y)$ with a much simpler term built with the Bernoulli numbers. Hence we expect that the computational complexity estimates of our approach will reveal an important reduction of the number of products needed to obtain the final result. In fact, this is the case and we show now how to prove it.

To compare the computational complexities of (3) and (8) we first need to choose $x > 0$, $y \in \mathbb{N}$, $y \geq 3$, $x(y - 1) < 1$ otherwise (3) is not applicable. In this case the main term in (3) requires the computation of one logarithm function and one product, while in (8) the computation of two logarithm functions, three products, and five sums are required.

Let now fix $\Delta \in (0, 1)$. Moreover let $m_1 := m_1(\Delta, x, y) \in \mathbb{N}$ and consider $D_{m_1}(x, y)$ as defined in (17). We can precompute and store the explicit formulae for $\phi_\ell(u)/(\ell(\ell - 1))$, $\ell \geq 2$; for example

$$\phi_2(u) = u^2 - u, \quad \phi_3(u) = u^3 - \frac{3}{2}u^2 + \frac{1}{2}u, \quad \phi_4(u) = u^4 - 2u^3 + u^2, \dots$$

Using Horner's scheme, see, e.g., Stoer and Bulirsch (2002), p. 317, the computation of a polynomial of a fixed degree ℓ requires ℓ products and ℓ sums. Then, for a fixed ℓ , to compute the summand in (17) we need $\ell + 1$ products and $\ell + 1$ sums because every power of x can be obtained using a repeated products strategy. Then the total cost of computing $D_{m_1}(x, y)$ is about $(m_1^2 + 3m_1)/2$ products and $(m_1^2 + 3m_1)/2$ sums. Moreover, to obtain a final result having a Δ -accuracy using (21), we can choose $m_1(\Delta, x, y) = \left\lceil \frac{|\log \Delta| + \log y}{|\log(xy)|} \right\rceil$, where as $\lceil u \rceil$ we have denoted the least integer greater than or equal to u .

Let now $m_2 := m_2(\Delta, x, y) \in \mathbb{N}$, and consider the sum $S_{m_2}(x, y)$ as defined in (6). In this case too we can precompute and store the values of $B_{2\ell}/(2\ell(2\ell - 1))$; for example

$$B_2 = \frac{1}{6}, \quad B_4 = -\frac{1}{30}, \quad B_6 = \frac{1}{42}, \quad B_8 = -\frac{1}{30}, \quad B_{10} = \frac{5}{66}, \dots$$

Then for a fixed ℓ , to compute the summand in (6) we need, using a repeated product strategy, four products and two sums. So the cost of computing $S_{m_2}(x, y)$ is $4m_2$ products and $2m_2$ sums. To obtain a final result having a Δ -accuracy using (9), we need to first evaluate the optimal value $\tilde{m}$ using (11) and then, if $E_{2\tilde{m}}(x) < \Delta$, choose the smallest $m_2 \leq \tilde{m}$ such that $E_{2m_2}(x) < \Delta$. Remark that, for any fixed $x > 0$, this

has to be done just once; moreover, the coefficients in (11) can be precomputed and stored.

In practice we will have that $m_2(\Delta)$ is smaller than $m_1(\Delta)$, for example see Table 1. Hence the computational cost in (6) is much smaller than the one in (3). We have also to consider that (3) is applicable only if $x(y-1) < 1$. This essentially means that if we need to compute $\log \Gamma(1/x + Y) - \log \Gamma(1/x)$ with $x(Y-1) \geq 1$, then the computation of (3) has to be repeated in every interval of the type $[1/\beta_i, 1/\beta_i + \gamma_i]$, where $i = 0, \dots, L$, $\beta_0 = x$, $1/\beta_L + \gamma_L = Y$, $\beta_i \gamma_i < 1$, $2 \leq Y/\max(\gamma_i) < L < Y/\min(\gamma_i)$ and L represents the total number of points used in the mesh; this is the mesh part of Yu-Shaw's algorithm. Hence the total computational cost of the mesh algorithm based on (3) becomes about Lm_1^2 products and Lm_1^2 sums which is much larger than the computational cost of (6). Moreover, the error accumulation phenomenon in the mesh algorithm based on (3) gives that the total error term is bounded by $(L+1)\Delta$, while the error term estimate in using (1) is Δ .

For the running times obtained in some experiments, see Sect. 5.

Remark 1 (On Bouneffouf (2020) preprint) Bouneffouf evaluated $\log \mathcal{L}(\mathbf{p}; \mathbf{x})$ by computing the paired difference of $\log \Gamma$ -values as a sum of logarithms, as in (15). In Section 4.1 of Bouneffouf (2020), he claims that this direct approach is faster than Yu-Shaw's mesh algorithm when $m > N$; but they have fixed $m = 20$ in their setting and hence Bouneffouf's claim might stand only for very small values of N . Unfortunately, his claim is completely unsubstantiated because in Figure 2 of Bouneffouf (2020) the running time of the mesh algorithm is misreported since microseconds (μs) are written as milliseconds (msec); compare with the corresponding Figure 6 of Yu and Shaw (2014). We also remark that in Table 1 of Bouneffouf (2020) the computational complexity of this direct approach is measured just using the number of performed evaluations of the logarithm function, while, for the other two methods there mentioned, Bouneffouf wrote the number of performed products. Moreover, again in Table 1 there, the computational cost of the mesh algorithm is misreported, since it should depend on x_k and the number of iterations should have been linked to the number of the points in the mesh.

5 Experiments

Unfortunately the code used by Yu and Shaw is not directly available; so to make a comparison between their methodology and ours we use the examples provided in their paper. In the following we will refer to the experiments described in Figures 5 and 6 of Yu and Shaw (2014) as fig5 and fig6, respectively, but we will compute at the same time $\log \mathcal{L}(\mathbf{p}; \mathbf{x})$, $F(\mathbf{p}; \mathbf{x})$ and $F^{(1)}(\mathbf{p}; \mathbf{x})$ while Yu and Shaw (2014) just computed the first of such three quantities. The case in fig5 represents a dataset whose main parameters are $K = 4$, $\psi = 1/200$, $\mathbf{p} = [1/10, 1/5, 3/10, 2/5]$, $\mathbf{x} = [n, n, n, n]$, where $n = 2500 \cdot a$, and $a \geq 1$ runs over the natural numbers. The case in fig6 represents a dataset whose main parameters are $K = 3$, $\psi = 1/60$,

$\mathbf{p} = [1/6, 1/3, 1/2]$, $\mathbf{x} = [n, 2n, 3n]$, where $n = 10^a$, and $a \geq 1$ runs over the natural numbers.

In these simulations, we are hence using exactly the same parameters they used in their paper. We compute exactly the same quantities and in Sect. 5.2 we will observe that our methodology is at least one order of magnitude faster. We recall that in Sect. 4 we already observed that our algorithm has a better computational complexity and provides a much better precision than theirs.

5.1 Multiprecision implementation (with Pari/GP)

As we described in Sect. 4.2, the computational complexity of evaluating $S_m(x, y)$ essentially does not depend on y but just on the choice of a suitable $m = m(x)$ to obtain a good accuracy (we further recall that if such an accuracy is not sufficient, it is always possible to increase it using the horizontal shift idea). This was experimentally confirmed with a Pari/GP script in which we replicated the computation contained in fig6 with an accuracy of 19 decimal digits ($m = 11$) for every $n = 10^a$, $a = 2, \dots, 50$ (y is equal to n in this setting). The running time for computing $\log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x})$, $F(\mathbf{p}, \psi; \mathbf{x})$ and $F^{(1)}(\mathbf{p}, \psi; \mathbf{x})$ with Corollary 1 and Theorems 2 and 3 was obtained by averaging the total time required to repeat 1000 times such a routine: the average time was less than 106 microseconds (μs) on a Dell OptiPlex-3050 machine (equipped with an Intel i5-7500 CPU, 3.40GHz, 16GB of RAM and running Ubuntu 20.04.3 LTS). For this problem the best possible accuracy is 26 decimal digits with $\tilde{m} = 31$; well beyond the typical precision required for these applications.

We also replicated the computation of fig5; in this case the best possible accuracy is 54 decimal digits with $\tilde{m} = 63$; again, well beyond the typical precision required for these applications. For every $n = 2500 \cdot a$, $a = 1, \dots, 10$, (y is equal to n in this setting) the average running time for computing $\log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x})$, $F(\mathbf{p}, \psi; \mathbf{x})$ and $F^{(1)}(\mathbf{p}, \psi; \mathbf{x})$ with Corollary 1 and Theorems 2 and 3 was 106 μs with a precision of 19 decimal digits ($m = 7$).

The verification of the accuracy of both the previous results was performed by recomputing the same quantities using the internal functions of Pari/GP using an accuracy of 100 decimal digits. For the case of fig6, the values of $\log \mathcal{L}(\mathbf{p}, \psi; \mathbf{x})$ and $F(\mathbf{p}, \psi; \mathbf{x})$ were confirmed for every $a = 2, \dots, 50$ while for $F^{(1)}(\mathbf{p}, \psi; \mathbf{x})$ we were able to verify them only for a up to 10. This is due to the fact that $F^{(1)}(\mathbf{p}, \psi; \mathbf{x})$ involves the *trigamma* function $F^{(1)}$ and, unfortunately, it seems that the Pari/GP implementation of $F^{(1)}$ is not optimised.³

³ In fact, Pari/GP does not have a direct implementation of $F^{(\ell)}$, $\ell \in \mathbb{N}^*$, but these functions can be obtained using the Hurwitz zeta function $\zeta(s, u)$, $s > 1$, $u > 0$, since $F^{(\ell)}(u) = (-1)^{\ell-1} (\ell!) \zeta(\ell + 1, u)$. So it is more precise to say that in Pari/GP the implementation of $\zeta(2, u)$ seems to be not optimised, at least for large values of u .

Table 2 Table of the averaged running times for the experiments

		Figure 5 of Yu and Shaw (2014)			Figure 6 of Yu and Shaw (2014)		
type	prec	n	N	Average time (μ s)	n	N	Average time (μ s)
double	3	1.750×10^5	$4n$	3.155	10^9	$6n$	0.164
double	5	1.675×10^5	$4n$	3.147	10^7	$6n$	0.186
double	10	na	na	na	10^2	$6n$	0.226
long double	3	1.750×10^5	$4n$	2.096	10^{13}	$6n$	0.513
long double	5	1.675×10^5	$4n$	2.081	10^{11}	$6n$	0.538
long double	10	1.500×10^5	$4n$	2.100	10^6	$6n$	0.565

5.2 C99 implementation

After having practically verified the correctness of our algorithm as described in Sect. 5.1, we implemented the same experiments using the C99 programming language, using both the `double` and the `long double` numerical types. In both these cases a well known additional difficulty arose: the floating point arithmetic can represent large numbers only losing accuracy in the mantissa. So, our implementation asks the user to insert the desired accuracy for the mantissa of the final result (we allowed this parameter $\text{prec} \in \{3, \dots, 10\}$) and then performs the computation until $|\log \mathcal{L}(\mathbf{p}, \boldsymbol{\psi}; \mathbf{x})|$ becomes too large to be represented without losing in the user-desired accuracy. We have to remark that this phenomenon was not mentioned at all in Yu and Shaw (2014).

In Table 2 we have collected the average running times obtained by repeating 1000 times the experiments described in fig5 and fig6 on the Dell Optiplex machine previously mentioned. The “na” code stands for “not available”: in this case the requirement about an accurate 10 decimal digits mantissa is too strong to be obtained with the `double` type.

We want to emphasize the result in the fourth row of Table 2 since it is comparable with the ones in Figure 6 of Yu and Shaw (2014). Using the `double` type, in the case $n = 10^7$, the average running time in Yu and Shaw (2014) was about 3μ s with no accuracy specified; with our algorithm we can certify at least five decimal digits with an average running time of 0.186μ s. However, we also have to consider that Yu and Shaw used a 4-core Intel Xeon E5630@2.53GHz (in fact they wrote 3.53GHz but we think this is a typo) which is considered to be about the 20% slower than our Intel i5-7500, 3.40GHz (released in 2017), see the following CPU-comparison. Hence, after having rescaled our result accordingly, we can say that our algorithm is, in this particular case, about 13.5-times faster than theirs; and it gives much more information since we computed $\log \mathcal{L}(\mathbf{p}, \boldsymbol{\psi}; \mathbf{x})$, $f(\mathbf{p}, \boldsymbol{\psi}; \mathbf{x})$ and $f^{(1)}(\mathbf{p}, \boldsymbol{\psi}; \mathbf{x})$ while Yu and Shaw (2014) obtained the first value only. If we would reduce our implementation to just compute $\log \mathcal{L}(\mathbf{p}, \boldsymbol{\psi}; \mathbf{x})$, it would be at least 30-times faster than theirs

(we did not try to optimise our code for determining the value of just one function instead of three).

This fits with the estimates in Sect. 4.2 in which we have shown that our algorithm requires a number of products that is essentially linear in the number of iterations and that Yu–Shaw’s one is essentially more than quadratic.

5.3 Larger cases

Combining what we have described in Sects 5.1 and 5.2, it is hence clear that if the parameters are not “too large” (it is not possible to be more precise about this since this depends on the choices of $(p, \psi; x)$ and `prec`), one can use the `double` or the `long double` C99-implementations. The remaining cases can be studied by inserting a multiprecision library in our C99 programs or using our Pari/GP script. In the current version of the code we have not introduced the horizontal shift as it was not needed for the experiments presented; however it can be added without requiring any major change in the code structure. Of course, at that point the time required for the horizontal shift should be included in the execution time measurements.

Acknowledgements The authors would like to thank Marco Vianello (Università di Padova) for having suggested one of the references. We would also like to thank the anonymous referees for their suggestions.

References

- Agresti A (2002) Categorical data analysis. Wiley series in probability and statistics, 2nd edn. Wiley, Hoboken
- Barnes E (1900) The theory of the gamma function. *Messenger Math.* 29:64–128
- Bouguila N (2008) Clustering of count data using generalized Dirichlet multinomial distributions. *IEEE Trans. Knowl. Data Eng.* 20:462–474
- Bouneffouf D (2020) Computing the Dirichlet-multinomial log-likelihood function. <http://arxiv.org/abs/2007.11967>
- Brown M et al (1993) Using Dirichlet mixture priors to derive hidden Markov models for protein families. In: *Proceedings of the first international conference on intelligent systems for molecular biology*. AAAI Press, pp 47–55
- Cameron CA, Trivedi PK (2013) Regression analysis of count data. *Econometric society monographs*, 2nd edn. Cambridge University Press, Cambridge
- Giudici P, Castelo R (2003) Improving Markov chain Monte Carlo model search for data mining. *Mach. Learn.* 50:127–158
- Hermite MCh (1895) Sur la fonction $\log \Gamma(a)$. *J. Reine Angew. Math.* 115:201–208
- Hooper PM (2004) Dependent dirichlet priors and optimal linear estimators for belief net parameters. In: *Proceedings of the 20th conference on Uncertainty in artificial intelligence*. AUAI Press, pp 251–259
- Leckenby JD, Kishi S (1984) The Dirichlet multinomial distribution as a magazine exposure model. *J. Mark. Res.* 21:100–106
- MacKay DJC, Bauman Peto LC (1994) A hierarchical Dirichlet language model. *Nat. Lang. Eng.* 1:1–19
- Madsen RE et al (2005) Modeling word burstiness using the Dirichlet distribution. In: *Proceedings of the 22nd international conference on machine learning*. ICML’05. ACM, New York, NY, pp 545–552

- Mimno D, McCallum A (2008) Topic models conditioned on arbitrary features with dirichlet-multinomial regression. In: Proceedings of the twenty-fourth conference on uncertainty in artificial intelligence (UAI2008), Helsinki, Finland
- Sjölander K et al (1996) Dirichlet mixtures: a method for improved detection of weak but significant protein sequence homology. *Comput. Appl. Biosci.* 12:327–45
- Stoer J, Bulirsch R (2002) Introduction to numerical analysis, 3rd edn. Springer, New York
- The PARI Group (2022) Pari/GP version 2.13.4. Bordeaux. <http://pari.math.u-bordeaux.fr>
- Winkelmann R (2008) Econometric analysis of count data, 5th edn. Springer, Berlin
- Yee TW (2010) The VGAM package for categorical data analysis. *J. Stat. Softw.* 32:1–34
- Yee TW (2022) VGAM: vector generalized linear and additive models. R package version 1.1-7. <https://cran.r-project.org/web/packages/VGAM/VGAM.pdf>
- Yee TW, Wild CJ (1996) Vector generalized additive models. *J. R. Stat. Soc. B* 58:481–493
- Yu P, Shaw C (2014) An efficient algorithm for accurate computation of the Dirichlet-multinomial log-likelihood function. *Bioinformatics* 30:1547–1554

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

REVISED PROOF